



NLTK — VS — spaCy

spaCy
INDUSTRIAL-STRENGTH
NLP IN PYTHON



Luiz Paludo



ITAI 2376



Houston Community College



August 2026

Introduction

- **NLTK** and **spaCy** are two popular Python libraries used for Natural Language Processing (NLP).
- Both libraries help computers understand, process, and analyze human language.
- They have different approaches, strengths, and are designed for different purposes.
- In this presentation, I will compare NLTK and spaCy based on their features, ease of use, performance, and applications.



Background

Different focus and purpose, same goal: working with human language



NLTK

Natural Language Toolkit

- Focuses on **education and research**.
- Provides many text corpora and linguistic resources.
- Widely used in **educational settings** because of its extensive documentation.
- Helps users **understand NLP concepts** and experiment with language data.

spaCy

INDUSTRIAL-STRENGTH
NLP IN PYTHON

- Focuses on **practical applications and real-world use**.
- Designed to analyze large amounts of text **quickly and efficiently**.
- Provides **pre-trained models** for common NLP tasks.
- Commonly used for **production environments**.



Key Features

Both libraries provide tools for common NLP tasks, but with different strengths.



NLTK

Natural Language Toolkit



Tokenization

Split text into words or sentences.



Stemming

Reduce words to their base form.



Tagging

Identify parts of speech.



Parsing

Analyze sentence structure.



Large collection of text corpora and linguistic resources



spaCy

INDUSTRIAL-STRENGTH
NLP IN PYTHON



Tokenization

Split text into words or sentences.



Part-of-speech tagging

Identify parts of speech.



Dependency parsing

Analyze grammatical relationships between words.



Named entity recognition

Identify people, organizations, locations, etc.



Pre-trained language models

Ready-to-use models for common NLP tasks.



Ease of Use and Flexibility

Both libraries are beginner friendly, but they offer different ways to work with NLP.



NLTK

Natural Language Toolkit

- Provides many **individual tools**, so users can understand each step of the process.
- Allows users to work with **each NLP task separately** (e.g., tokenization, stop words, stemming).
- Good for **learning and experimenting** with different NLP techniques.
- Offers a lot of **flexibility** for research and custom projects.

spaCy

INDUSTRIAL-STRENGTH
NLP IN PYTHON

- Provides **complete processing pipelines** that can perform multiple NLP tasks at the same time.
- Designed to be **easy to use**, with pre-trained models.
- Allows advanced users to **customize pipelines** and train models for specific projects.
- **Simplifies** the process of building NLP applications.



NLTK gives **more control** with individual tools, while spaCy offers a **more complete** and **simplified approach**.



Example of Code

Here is a simple example of how to use each library to perform the same task: tokenization and named entity recognition (NER) on a short text.



NLTK Example

Tokenization and Named Entity Recognition

```
python
# Import NLTK and download required data
import nltk
from nltk.tokenize import word_tokenize
from nltk import ne_chunk, pos_tag

# Download resources (only need to do this once)
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
nltk.download('maxent_ne_chunker')
nltk.download('words')

# Example text
text = "Apple is investing in AI in the United States."

# Tokenization
tokens = word_tokenize(text)

# Part-of-speech tagging and Named Entity Recognition
pos_tags = pos_tag(tokens)
ner_tree = ne_chunk(pos_tags)

print("Tokens:", tokens)
print("\nNamed Entities:")
for chunk in ner_tree:
    if hasattr(chunk, 'label'):
        print(chunk.label(), ' '.join(c[0] for c in chunk))
```

Copy code

Output (NLTK)

```
Tokens:
['Apple', 'is', 'investing',
'in', 'AI', 'in', 'the',
'United', 'States', '.']

Named Entities:
ORGANIZATION Apple
GPE United States
```

What this does:

- Uses NLTK to tokenize the text.
- Performs part-of-speech tagging.
- Identifies named entities (e.g., organizations, locations).
- Shows the tokens and named entities found in the text.

spaCy Example

Tokenization and Named Entity Recognition

```
python
# Import spaCy and load the model
import spacy

# Load English model
nlp = spacy.load("en_core_web_sm")

# Example text
text = "Apple is investing in AI in the United States."

# Process the text
doc = nlp(text)

# Tokenization
print("Tokens:", [token.text for token in doc])

# Named Entity Recognition
print("\nNamed Entities:")
for ent in doc.ents:
    print(ent.label_, ent.text)
```

Copy code

Output (spaCy)

```
Tokens:
['Apple', 'is', 'investing',
'in', 'AI', 'in', 'the',
'United', 'States', '.']

Named Entities:
ORG Apple
GPE United States
```

What this does:

- Uses spaCy to process the text.
- Automatically performs tokenization, part-of-speech tagging, and named entity recognition.
- Shows the tokens and named entities found in the text.



Both libraries can perform the same task, but with different approaches and levels of complexity.



Performance and Applications

Both libraries are effective for NLP, but they are commonly used in different scenarios.



NLTK

Natural Language Toolkit



Performance

Slower compared to spaCy, especially with large datasets.



Common Applications

Used in educational and research projects.



Best For

Small to medium projects and learning NLP concepts.



Examples

Text analysis, language research, and academic projects.

spaCy

INDUSTRIAL-STRENGTH
NLP IN PYTHON



Performance

Faster and more efficient, designed for large-scale text processing.



Common Applications

Used in real-world and production applications.



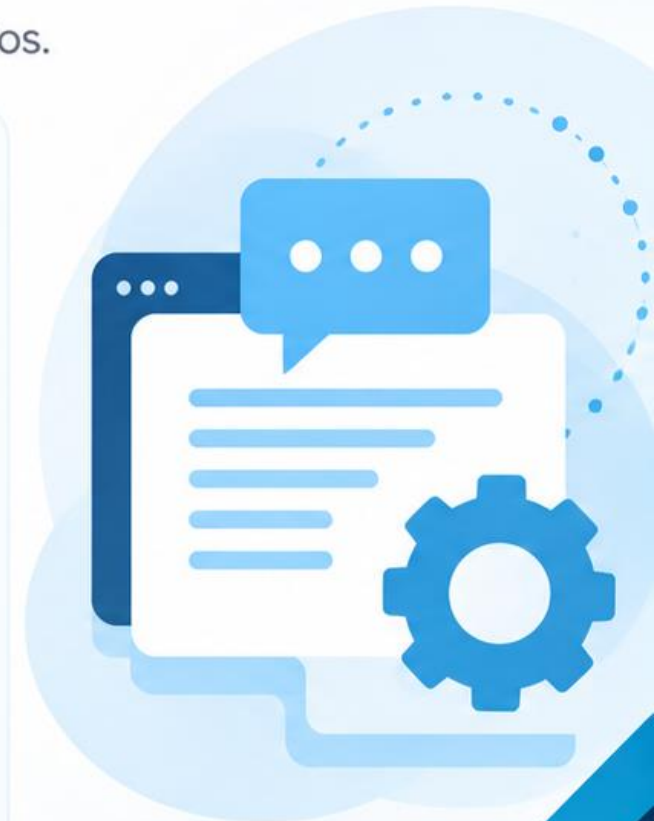
Best For

Large projects and production environments.



Examples

Information extraction, text classification, and named entity recognition.



Conclusion

Both NLTK and spaCy are powerful NLP libraries, but the best choice depends on your goals.



NLTK

Natural Language Toolkit

Choose NLTK if you want:

- ✓ To **learn** NLP concepts.
- ✓ More **educational resources** and text corpora.
- ✓ To **experiment** with individual NLP techniques.

spaCy

INDUSTRIAL-STRENGTH
NLP IN PYTHON

Choose spaCy if you want:

- ✓ **Fast** NLP processing.
- ✓ **Pre-trained models** for common NLP tasks.
- ✓ To build **practical, real-world applications**.



Bottom Line:

Both NLTK and spaCy are great NLP libraries. The right choice depends on your experience, goals, and the type of project you want to work on.





NLTK

Natural Language Toolkit

Thank You for Your Time!



Any Questions?

spaCy

INDUSTRIAL-STRENGTH
NLP IN PYTHON



Luiz Paludo



ITAI 2376



Houston Community College



August 2026